

zmk-config-roBa

キーマップ設定ガイド

roBa（トラックボール付き分割キーボード）用 ZMK 設定リポジトリの読み方・育て方

ZMK Studio は試す場所、GitHub の `roBa.keymap` は残す場所。

README.md をスライド形式にまとめた資料です。詳細は README.md 本文を参照してください。

この資料の構成

Part 1 — 基礎と運用方針

1. リポジトリ構成
2. 運用方針
3. 初回セットアップ

Part 2 — キーマップを編集する

4. Keymap Editor の使い方
5. `.keymap` の基本概念
6. 機能の使い方と具体例

Part 3 — roBa 固有の設定と書き込み

7. 現在のキーマップ構成
8. トラックボールの設定
9. ビルドと書き込み

Part 4 — 日々の運用と困ったとき

10. ZMK Studio との付き合い方
11. 設定を育てる運用 Tips
12. トラブルシューティング
13. チートシート
14. 参考資料

現在のキーマップ（`keymap-drawer/roBa.svg`）



上は default レイヤーのみ。全レイヤーの図は `keymap-drawer/roBa.svg` を参照。Actions の **Draw Keymap** で再生成できます（11.5 節）。

PART 1

基礎と運用方針

1. リポジトリ構成 ／ 2. 運用方針 ／ 3. 初回セットアップ

1. リポジトリ構成

```
zmk-config-roBa/
├── config/
│   ├── roBa.keymap    ← 最重要: キーマップ本体
│   ├── roBa.json      ← Keymap Editor 用の物理レイアウト
│   └── west.yml       ← ZMK 本体と PMW3610 ドライバの取得元
├── boards/shields/roBa/
│   ├── roBa.dtsi      ← 物理レイアウト・マトリクス (通常触らない)
│   ├── roBa_R.overlay / roBa_R.conf ← 右 (トラックボール側)
│   └── roBa_L.overlay / roBa_L.conf ← 左 (エンコーダ側)
├── build.yaml         ← Actions でビルドするターゲット一覧
├── keymap-drawer/     ← キーマップ図 (roBa.svg / roBa.yaml)
├── docs/slides/       ← この資料 (Marp)
└── .github/workflows/
    ├── build.yaml     ← push ごとにファームウェアをビルド
    ├── draw.yaml      ← キーマップ図を再生成 (手動)
    └── slides.yaml    ← この資料を PDF 化
```

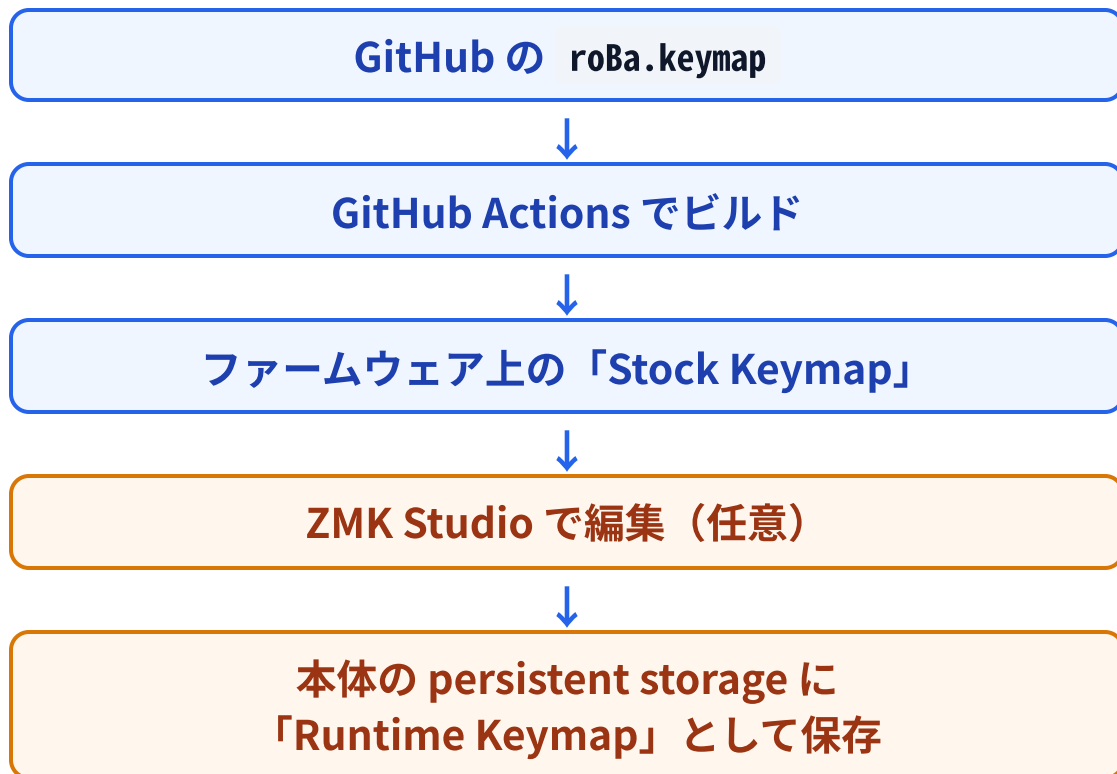
普段編集するのは3つだけ

ファイル	役割
roBa.keymap	キー割り当て、レイヤー、Combo、Macro、Auto Mouse
roBa_R.conf	CPI、Auto Mouse timeout、スクロール感度
build.yaml	ビルド対象 (通常は変更不要)

NOTE

roBa.dtsi や .overlay はハード定義なので通常は触りません。

設定の 2 層構造



roBa の設定には 2 種類あります。

- **ファームウェアに焼き込まれる設定** (Stock Keymap)
- **ZMK Studio で本体に保存される設定** (Runtime Keymap)

注意

一度 ZMK Studio でキーマップを触ると、その後 `.keymap` を変更して再ビルド・書き込みしても **Studio 側の保存済み設定が優先** されます。

`.keymap` の内容に戻すには ZMK Studio の **Restore Stock Settings** が必要です (10 章)。

2. 運用方針 — GitHub の `main` が正本



ツール	用途	推奨度
GitHub + roBa.keymap	正式な設定・バックアップ・変更履歴	◎
Keymap Editor	roBa.keymap を GUI で編集して GitHub に直接 commit	◎
手書き編集	Combo / Macro / カスタム Behavior / .conf などの高度な編集	○
ZMK Studio	一時的な試行・動作確認（正本にはしない）	△

TIP

原則: 「手元の roBa がどうなっているか」ではなく「GitHub の `main` に何が書かれているか」を正式な設定とみなす。

故障・紛失・買い替えがあっても同じ操作体系を再現できます。

3. 初回セットアップ

3.1 公式リポジトリを Fork

1. GitHub にログイン
2. <https://github.com/kumamuk-git/zmk-config-roBa> を開く
3. 右上の **Fork** → **Create fork**
4. <あなたのユーザー名>/zmk-config-roBa を以降編集する

3.2 Actions を有効化

Fork 直後は Actions が無効なことがあります。

1. 自分の `zmk-config-roBa` を開く
2. **Actions** タブを開く
3. `I understand my workflows, go ahead and enable them` をクリック

3.3 ローカルにも Clone (推奨)

GitHub 上と Keymap Editor だけでも運用できますが、`.conf` の編集や Tag 付けをするならローカル Clone があると便利です (下のコマンド)。

```
git clone https://github.com/<YOUR_GITHUB_ID>/zmk-config-roBa.git
cd zmk-config-roBa
git remote add upstream https://github.com/kumamuk-git/zmk-config-roBa.git # 公式更新を取り込みやすくする
git remote -v # origin → 自分の Fork / upstream → roBa 公式
```


3.4 すでに ZMK Studio で設定している場合

注意

いきなり **Restore Stock Settings** を押さないでください。Studio から `.keymap` へ Export する機能はまだありません。

1. ZMK Studio の **全レイヤー**をスクリーンショットで記録する
(Key Press / Mod-Tap / Layer-Tap / Mouse Key / Bluetooth / Transparent と None の区別)
2. **レイヤー名と番号** も記録する (番号は Auto Mouse などから参照される)
3. Keymap Editor で `roBa.keymap` に同じ配置を再現する
4. Save (= GitHub に commit)
5. GitHub Actions のビルド成功を確認
6. UF2 を書き込む
7. 最後に ZMK Studio で **Restore Stock Settings**
8. GitHub 側の設定が反映されたことを確認

NOTE

先に GitHub 側を完成させてから Restore する、という順番が重要です。

PART 2

キーマップを編集する

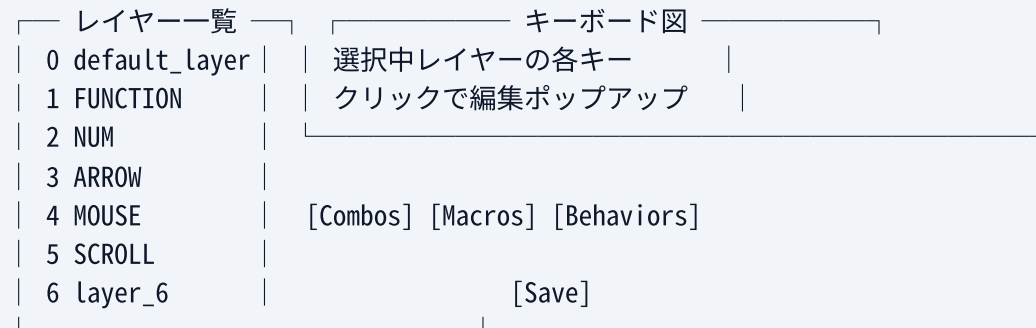
4. Keymap Editor の使い方 / 5. `.keymap` の基本概念 / 6. 機能の使い方と具体例

4. Keymap Editor — ブラウザで .keymap を GUI 編集

4.1 自分の Fork を接続する

1. Keymap Editor を開く
2. 上部のタブで **GitHub** を選ぶ
3. **Login with GitHub** を押す
4. **Authorize Keymap Editor** で認可する
5. GitHub App のインストール画面で自分の `zmk-config-roBa` へのアクセスを許可（`Only select repositories` で絞ってよい）
6. **Repository** で `zmk-config-roBa`、**Branch** で `main` を選ぶ
7. `config/roBa.keymap` が読み込まれ、キーボードの図が表示される

4.2 画面の見方



- URL: <https://nickcoutsos.github.io/keymap-editor/>

NOTE

物理レイアウトは `config/roBa.json` から読み込まれます。消したり改名すると Editor がレイアウトを見つけれません。

4.3 キーを変更する

1. 変更したいキーをクリックする
2. 左側で **Behavior** を選ぶ
(`&kp` `&mt` `<` `&mo` `&to` `&tog` `&mkp` `&bt` `&trans` `&none` など)
3. 右側でパラメータを選ぶ
 - `&kp` → キーコード一覧から選ぶ (検索欄に `ENTER` や `F5` と入力すると絞り込める)
 - `&mt` → 「修飾キー」と「タップ時のキー」の 2 つ
 - `<` → 「レイヤー」と「タップ時のキー」の 2 つ
4. 修飾キー付き (例: `Ctrl+Shift+Tab`) は、キーコードを選んだ後に **modifier** のトグル (`LS` `LC` `LA` `LG`) を重ねる
5. ポップアップを閉じると図に反映される (まだ GitHub には保存されていない)

TIP

手書きで定義したカスタム Behavior (この repo なら `<_to_layer_0` と `&to_layer_0`) も Behavior 一覧に出ています。

NOTE

キーは「Behavior + パラメータ」の組み合わせです。 `&mt` / `<` の考え方は 6.1 / 6.2 節を参照。

4.4 レイヤーの追加・名前変更 / 4.5 Combo の編集

4.4 レイヤーを追加・名前変更・並べ替え

- レイヤー一覧の **+** で新しいレイヤーを末尾に追加
- レイヤー名をクリックすると名前を変更（`.keymap` のノード名になる）
- 並べ替え・削除もレイヤー一覧のボタンから

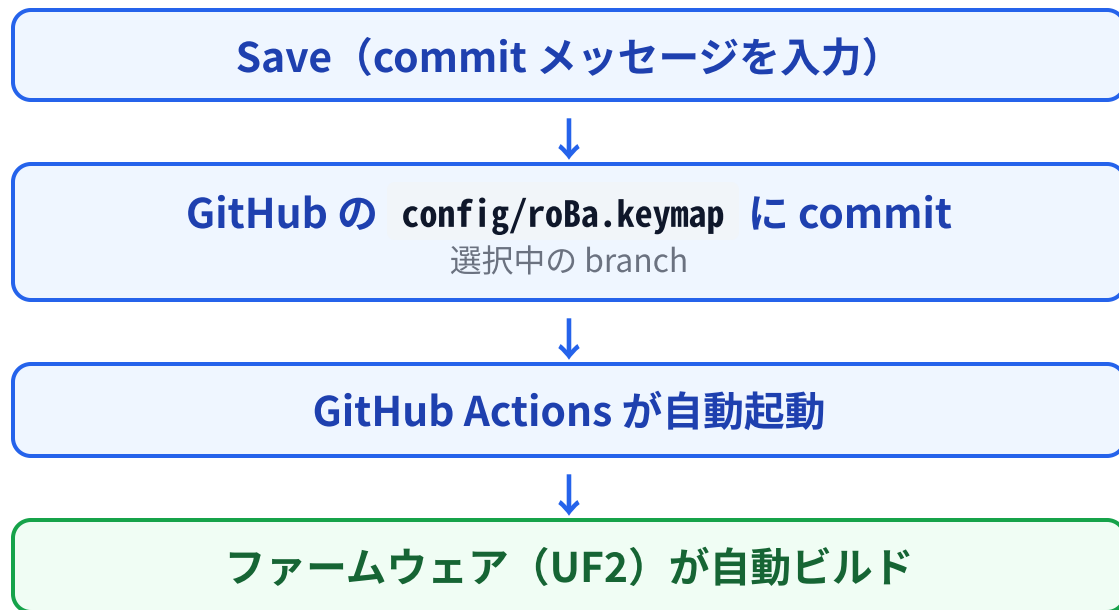
注意

レイヤー番号は `.keymap` に書かれた順番で決まります。 途中に挿入すると `< 2 SPACE` や `automouse-layer = <4>` が別のレイヤーを指します。新しいレイヤーは **末尾に追加** が安全です（5.3 節）。

4.5 Combo を編集する

1. **Combos** パネルを開く
 2. 既存の Combo を選ぶか **+** で追加
 3. 同時押しするキーを図上でクリック（`key-positions` に対応）
 4. 発動する Behavior を設定（`bindings` に対応）
- `timeout-ms` や `layers` のような細かい指定は手書きの方が確実です（6.7 節）。

4.6 Save の意味 — Save = commit



Keymap Editor の **Save** は、単なるブラウザ内保存ではありません。

- Save した時点で **変更履歴が GitHub に残る**
- ZMK Studio との最大の違いはここ

NOTE

Editor は Save 時に `.keymap` を整形し直します。手書きの `&trackball { ... } / behaviors { ... } / macros { ... }` / コメントは保持されますが、インデントや列揃えは変わることがあります。

4.7 Editor でできること・できないこと

やりたいこと	Editor	手書き
キーの割り当て変更	◎	○
Mod-Tap / Layer-Tap の設定	◎	○
レイヤーの追加・名前変更	◎	○
Combo の追加・変更	○	◎
Macro の作成	△	◎
Hold-Tap の細かい設定 (tapping-term-ms など)	×	◎
カスタム Behavior (Home Row Mods など)	×	◎
<code>&trackball { automouse-layer ... }</code>	×	◎
<code>.conf</code> (CPI / timeout)	×	◎

TIP

Editor で編集した `.keymap` に、あとから手書きで Behavior を追加しても問題ありません。**両方を組み合わせて使えます。**

- 日常のキー変更 → **Editor**
- Combo / Macro / Hold-Tap 調整 / トラックボール設定 → **手書き**

5.1 .keymap ファイルの全体像

```
#include <behaviors.dtsi>           // 標準 Behavior (&kp, &mt, &lt ... )
#include <dt-bindings/zmk/bt.h>      // BT_SEL などの定数
#include <dt-bindings/zmk/keys.h>    // キーコード (A, ENTER, F1 ...)
#include <dt-bindings/zmk/pointing.h> // MB1 などマウス関連の定数

&mt { ... };                       // 標準 Behavior の挙動を変える (任意)
&trackball { ... };                // トラックボール: Auto Mouse / Scroll レイヤーの指定

/ {
    combos { ... };                // 同時押し
    macros { ... };                // マクロ
    behaviors { ... };              // カスタム Behavior

    keymap {
        compatible = "zmk,keymap";

        default_layer { bindings = < ... >; }; // Layer 0
        FUNCTION      { bindings = < ... >; }; // Layer 1
        ...
    };
};
```

.keymap を読めると「Editor が何をしているか」が分かり、Combo や Macro を安心して追加できます。

- 上部の `#include` で Behavior・キーコード・定数を取り込む
- `&mt { }` `&trackball { }` は **既存ノードの上書き**
- `/ { }` の中に Combo / Macro / Behavior / keymap を書く
- `keymap { }` 内の **書いた順番** がレイヤー番号になる

5.2 Behavior とキーコード

各キーは **Behavior**（何をするか）と **パラメータ** の組み合わせです。

```
&kp A           // Key Press: A
&kp LS(A)       // Shift + A（修飾キーは関数のように重ねる）
&kp LC(LS(TAB)) // Ctrl + Shift + Tab
```

修飾キー関数

関数	意味
LS() / RS()	左 / 右 Shift
LC() / RC()	左 / 右 Ctrl
LA() / RA()	左 / 右 Alt
LG() / RG()	左 / 右 GUI (Windows / Command)

よく使う Behavior (1/2)

表記	意味	例
&kp	Key Press	&kp A
&mt	Mod-Tap（長押しで修飾、タップでキー）	&mt LEFT_SHIFT Z
<	Layer-Tap（長押しでレイヤー、タップでキー）	< 2 SPACE
&mo	Momentary Layer（押している間だけ）	&mo 3
&to	指定レイヤーに切り替え（戻らない）	&to 0
&tog	レイヤーのトグル	&tog 6
&sk	Sticky Key（次の 1 キーだけ修飾）	&sk LSHIFT

5.2 よく使う Behavior (2/2)

表記	意味	例
<code>&sl</code>	Sticky Layer (次の 1 キーだけレイヤー)	<code>&sl 2</code>
<code>&trans</code>	下位レイヤーを透過	<code>&trans</code>
<code>&none</code>	何もしない (下位も無視)	<code>&none</code>
<code>&mkp</code>	Mouse Key Press (クリック)	<code>&mkp MB1</code>
<code>&bt</code>	Bluetooth 操作	<code>&bt BT_SEL</code> <code>0</code>
<code>&bootloader</code>	ブートローダーに入る (書き込みモード)	<code>&bootloader</code>
<code>&caps_word</code>	次の単語だけ大文字	<code>&caps_word</code>

NOTE

キーコードの一覧:

<https://zmk.dev/docs/keymaps/list-of-keycodes>

- `&trans` と `&none` の違いは 5.3 節
- マウス系 (`&mkp` `&msc` `&mmv`) は 6.5 節
- Bluetooth / bootloader は 6.6 節

5.3 レイヤーとレイヤー番号

ZMK では `keymap { }` 内に書かれた **順番** でレイヤー番号が決まります。名前は人間向けのラベルで、番号指定には使えません。

```
default_layer → 0
FUNCTION      → 1
NUM           → 2
ARROW         → 3
MOUSE         → 4
SCROLL        → 5
layer_6       → 6
```

レイヤーは重なって動作します。上位レイヤーが有効なとき、`&trans` のキーは下のレイヤーの割り当てが使われます。

Layer 4 (MOUSE):	trans	trans	MB1	MB3	MB2	trans
Layer 0 (default):	H	J	K	L	'	...
実際の動作:	H	J	MB1	MB3	MB2	'

TIP

`&none` は「何も起きない」で、下のレイヤーも見ません。
Auto Mouse 中の文字誤入力を防ぎたい場合に使います。

5.4 キー位置番号（Combo で使う）

Combo は「どのキーとどのキーを同時に押すか」を **位置番号** で指定します。
左上から右へ、行ごとに振られます（**0 始まり、全 43 キー**）。

位置番号

0	1	2	3	4			5	6	7	8	9	
10	11	12	13	14	15		16	17	18	19	20	21
22	23	24	25	26	27		28	29	30	31	32	33
34	35	36	37	38	39		40	41				42

現在の default レイヤー

Q	W	E	R	T			Y	U	I	O	P
A	S	D	F	G	⌘⇧S	-	H	J	K	L	'
Z	X	C	V	B	:	;	N	M	,	.	/
Ctrl	Win	Alt	変換	SpC	無変換	BS	Ent				Del

NOTE

Home Row Mods の `hold-trigger-key-positions`（6.1 節）や Combo の `key-positions`（6.7 節）はこの番号を使います。

6.1 Mod-Tap (`&mt`) — 長押しで修飾キー、タップでキー

```
&mt LEFT_SHIFT Z    // 長押し: Shift / タップ: z
&mt LCTRL ESCAPE    // 長押し: Ctrl  / タップ: Esc
&mt LGUI TAB        // 長押し: Win   / タップ: Tab
```

Keymap Editor では Behavior に `&mt` を選び、1 つ目に修飾キー、2 つ目にタップ時のキーを指定します。

この repo では **Z キー**が `&mt LEFT_SHIFT Z` になっており、左手小指を動かさずに Shift が押せます。

判定を調整する (この repo の設定)

```
&mt {
  flavor = "balanced";
  quick-tap-ms = <0>;
};
```

プロパティ	意味	既定値
tapping-term-ms	この時間より長く押すと「長押し」扱い	200
flavor	長押し / タップの判定方法 (次スライド)	hold-preferred
quick-tap-ms	この時間内に同じキーを再度押すと必ずタップ扱い (連打・リピート用)。0 や未設定で無効	無効
require-prior-idle-ms	直前のキー入力からこの時間以内なら必ずタップ扱い (速いタイピング中の誤爆防止)	無効

TIP

Shift のつもりが z → tapping-term-ms を短く / z のつもりが Shift → 長く

6.1 Hold-Tap の flavor

flavor	挙動
hold-preferred	tapping-term-ms を超えるか、他のキーが押されたら即「長押し」
balanced	tapping-term-ms を超えるか、他のキーが押されて 離されたら 「長押し」。バランス型でおすすめ
tap-preferred	tapping-term-ms を超えたときだけ「長押し」。他のキーを押しても影響しない
tap-unless-interrupted	他のキーが押されない限りタップ。押されたら長押し

誤爆の傾向と対処

- 長押しのつもりがタップになる → tapping-term-ms を短く、hold-preferred を試す
- タップのつもりが長押しになる → tapping-term-ms を長く、require-prior-idle-ms を設定

連打がリピートしない

- quick-tap-ms = <150>; のように設定すると、同じキーの素早い再押下がタップ（＝キーリピート）になる

6.1 具体例: Home Row Mods

ホームポジション（A S D F / J K L ;）に修飾キーを重ねる定番構成。**反対側の手で押したときだけ長押し**と判定するカスタム Behavior で誤爆を減らします。

```
/ {
  behaviors {
    // 左手用: 右手側のキーと組み合わせたときだけ長押し
    hml: home_row_mod_left {
      compatible = "zmk,behavior-hold-tap";
      #binding-cells = <2>;
      bindings = <&kp>, <&kp>;
      flavor = "balanced";
      tapping-term-ms = <220>;
      quick-tap-ms = <150>;
      require-prior-idle-ms = <100>;
      hold-trigger-key-positions = <5 6 7 8 9 16 17 18 19 20 21 28 29 30 31 32 33 40 41 42>;
      hold-trigger-on-release;
    };
    // 右手用 hmr は左手側の位置番号
    // <0 1 2 3 4 10 11 12 13 14 15 22 23 24 25 26 27 34 35 36 37 38 39> を指定
    hmr: home_row_mod_right {
      compatible = "zmk,behavior-hold-tap";
      #binding-cells = <2>;
      bindings = <&kp>, <&kp>;
      flavor = "balanced";
      tapping-term-ms = <220>;
      quick-tap-ms = <150>;
      require-prior-idle-ms = <100>;
      hold-trigger-key-positions = <0 1 2 3 4 10 11 12 13 14 15 22 23 24 25 26 27 34 35 36 37 38 39>;
      hold-trigger-on-release;
    };
  };
};
```

hold-trigger-key-positions には 5.4 節の位置番号を使います。

default レイヤーの該当キーを置き換えます。

```
&hml LGUI A    &hml LALT S
&hml LCTRL D   &hml LSHIFT F
&hmr RSHIFT J  &hmr RCTRL K
&hmr RALT L    &hmr RGUI SQT
```

TIP

&hml / &hmr は Keymap Editor の Behavior 一覧にも現れるので、パラメータ変更は GUI からできます。

6.2 Layer-Tap (<lt;) — 長押しでレイヤー、タップでキー

親指キーの定番です。Editor では <lt; を選び、1 つ目にレイヤー、2 つ目にタップ時のキーを指定します。

```
<lt; 2 SPACE // 長押し: Layer 2 (NUM) / タップ: Space
<lt; 1 ENTER // 長押し: Layer 1 (FUNCTION) / タップ: Enter
<lt; 5 I      // 長押し: Layer 5 (SCROLL) / タップ: i
```

<mt と同様に <lt; { tapping-term-ms = <180>; }; など
で判定を調整できます。

具体例: この repo のカスタム Layer-Tap

日本語入力用に「長押しでレイヤー、タップで
Layer 0 に戻ってから 変換 / 無変換を送る」
Behavior を定義しています。

```
<lt_to_layer_0 6 INT_HENKAN // 長押し: Layer 6 / タップ: Layer 0 に戻して 変換
<lt_to_layer_0 3 INT_MUHENKAN // 長押し: Layer 3 / タップ: Layer 0 に戻して 無変換
```

```
macros {
  to_layer_0: to_layer_0 {
    compatible = "zmk,behavior-macro-one-param";
    #binding-cells = <1>;
    bindings = <&to 0 &macro_param_1to1 &kp MACRO_PLACEHOLDER>;
  };
};

behaviors {
  lt_to_layer_0: lt_to_layer_0 {
    compatible = "zmk,behavior-hold-tap";
    #binding-cells = <2>;
    bindings = <&mo>, <&to_layer_0>;
    // 長押し: &mo <layer> / タップ: &to_layer_0 <key>
    tapping-term-ms = <200>;
  };
};
```

TIP

<to で別のレイヤーに固定した後でも、このキー
をタップすれば必ず default に戻れます。

6.3 レイヤー切り替え / 6.4 Sticky Key

6.3 &mo / &to / &tog / &sl

```
&mo 3    // 押している間だけ Layer 3
&to 6    // Layer 6 に切り替え（戻すには別のキーで &to 0）
&tog 6   // 押すたびに Layer 6 の ON / OFF を切り替え
&sl 2    // 次の 1 キーだけ Layer 2 (Sticky Layer)
```

注意

&to で移動したレイヤーには、**戻るためのキーを必ず置いて**ください。この repo では &to_layer_0 を親指のタップに割り当てて逃げ道にしています。

6.4 Sticky Key (&sk)

次の 1 キーだけ修飾を効かせます。「Shift を押しながら」が苦手な位置の修飾キーに便利です。

```
&sk LSHIFT // 押して離し、次に a を押すと A
&sk LC(LALT) // 次の 1 キーに Ctrl+Alt を付ける
```

6.5 Mouse Key / 6.6 Bluetooth と bootloader

6.5 &mkp / &msc / &mmv

#include <dt-bindings/zmk/pointing.h> と
CONFIG_ZMK_POINTING=y (この repo では設定済み)
が必要です。

```
&mkp MB1 // 左クリック
&mkp MB2 // 右クリック
&mkp MB3 // 中クリック
&mkp MB4 // 戻る
&mkp MB5 // 進む
&msc SCRL_UP // ホイール上
&msc SCRL_DOWN // ホイール下
&mmv MOVE_UP // マウスカーソル移動 (キーで動かす場合)
```

MOUSE レイヤー (Layer 4) の J / K / L に MB1 / MB3 / MB2 を置いてあり、トラックボールを動かすと自動で有効になります (8 章)。

6.6 &bt / &bootloader

```
&bt BT_SEL 0 // プロファイル 0 に切り替え (0~4)
&bt BT_CLR // 現在のプロファイルのペアリングを削除
&bt BT_CLR_ALL // 全プロファイルのペアリングを削除
&bootloader // 書き込みモードに入る (リセットボタン不要)
```

TIP

誤操作を避けるため、専用の SYSTEM レイヤー (この repo では Layer 6) に置くのが定番です。

6.7 Combo（同時押し）

```
/ {
  combos {
    compatible = "zmk,combos";

    tab {
      bindings = <&kp TAB>;
      key-positions = <11 12>; // S + D
    };
    shift_tab {
      bindings = <&kp LS(TAB)>;
      key-positions = <12 13>; // D + F
    };
    // レイヤーを限定したり、判定時間を変えたりする例
    esc_on_base {
      bindings = <&kp ESCAPE>;
      key-positions = <1 2>; // W + E
      layers = <0>; // Layer 0 のときだけ有効
      timeout-ms = <40>; // 同時押しと見なす時間（既定 50ms）
      require-prior-idle-ms = <100>; // 速いタイピング中は発動しない
    };
  };
};
```

位置番号は 5.4 節を参照してください。

現在定義されている Combo

同時押し	位置	出力
S + D	11, 12	Tab
D + F	12, 13	Shift + Tab
A + S	10, 11	Layer 0 に戻して 無変換
L + '	20, 21	"
C + V	24, 25	=

6.8 Macro（マクロ） — 複数のキー操作を 1 キーに

```

/ {
  macros {
    // "hello" と打つ
    hello: hello {
      compatible = "zmk,behavior-macro";
      #binding-cells = <0>;
      bindings = <&kp H &kp E &kp L &kp L &kp 0>;
    };
    // Ctrl を押したまま C → V を送る（押す / 離すを明示する例）
    ctrl_c_v: ctrl_c_v {
      compatible = "zmk,behavior-macro";
      #binding-cells = <0>;
      wait-ms = <30>;    // 各操作の間隔
      tap-ms = <30>;     // キーを押している時間
      bindings
        = <&macro_press &kp LCTRL>
          , <&macro_tap &kp C &kp V>
          , <&macro_release &kp LCTRL>;
    };
  };
};

```

- キーマップ側では `&hello` や `&ctrl_c_v` として使う
- `¯o_press` / `¯o_tap` / `¯o_release` で押す・離すを明示できる
- パラメータを受け取るマクロ
（`zmk,behavior-macro-one-param`）の例は 6.2 節の `to_layer_0`

6.9 ロータリーエンコーダ（sensor-bindings）

左側のロータリーエンコーダの動作は **レイヤーごと** に `sensor-bindings` で指定します。

```
default_layer {  
    bindings = < ... >;  
    sensor-bindings = <&inc_dec_kp PG_UP PAGE_DOWN>;  
    // 回転で PageUp / PageDown  
};  
  
ARROW {  
    bindings = < ... >;  
    sensor-bindings = <&inc_dec_kp LC(PAGE_UP) LC(PAGE_DOWN)>;  
    // タブ切り替え  
};
```

- `sensor-bindings` を書かないレイヤーでは **下位レイヤーの設定** が使われる
- 音量にしたい場合:

```
sensor-bindings = <&inc_dec_kp C_VOLUME_UP C_VOLUME_DOWN>;
```

6.10 具体例: よくある変更 (1/2)

Backspace と Delete を入れ替える

```
// default_layer の親指行 (変更前)
&kp BACKSPACE &lt; 1 ENTER &kp DEL
// 変更後
&kp DEL      &lt; 1 ENTER &kp BACKSPACE
```

Space の長押しを NUM ではなく ARROW にする

```
&lt; 2 SPACE → &lt; 3 SPACE
```

Esc を Ctrl との Mod-Tap にして左上に置く

```
&kp Q → &mt LCTRL Q // タップ: q / 長押し: Ctrl
```

TIP

どれも Keymap Editor で該当キーをクリックして Behavior とパラメータを変えるだけで実現できます (4.3 節)。

6.10 具体例: よくある変更 (2/2) — 新しいレイヤーを末尾に追加

```
keymap {
    ...
    layer_6 { ... };

    // 新規: Layer 7
    MEDIA {
        bindings = <
        &trans &trans &trans &trans &trans          &kp C_PREV &kp C_PP &kp C_NEXT &trans &trans
        &trans &trans &trans &trans &trans &trans &trans &kp C_VOL_DN &kp C_MUTE &kp C_VOL_UP &trans &trans
        &trans &trans &trans &trans &trans &trans &trans &trans &trans &trans &trans &trans &trans
        &trans &trans &trans &trans &trans &trans &trans &trans &trans &trans &trans &trans &trans
        >;
    };
};
```

- 追加後、どこかのキーに `&mo 7` や `< 7 XXX` を置いて呼び出します
- 各レイヤーの `bindings` は **43 個** ちょうどにする（多くても少なくともビルドが失敗する）
- 途中に挿入せず **末尾に追加** する（4.4 / 5.3 節）

PART 3

roBa 固有の設定と書き込み

7. 現在のキーマップ構成 / 8. トラックボールの設定 / 9. ビルドと書き込み

7. 現在のキーマップ構成

Layer	名前	役割	呼び出し方
0	default_layer	通常の文字入力	常時
1	FUNCTION	F1～F13	Enter 長押し (< 1 ENTER)
2	NUM	テンキー配置の数字と記号	Space 長押し (< 2 SPACE)
3	ARROW	矢印、Home / End、タブ切り替え、Esc	無変換 長押し (<_to_layer_0 3)
4	MOUSE	J / K / L に 左 / 中 / 右クリック	トラックボールを動かすと自動 (Auto Mouse)
5	SCROLL	このレイヤー中はトラックボールがスクロール	I 長押し (< 5 I)
6	layer_6	Bluetooth 切り替え、 &bootloader 、 BT_CLR	変換 長押し (<_to_layer_0 6)

- Z 長押しで Shift (&mt LEFT_SHIFT Z)、NUM レイヤーでは 0 長押しで Shift
- 左手親指行の内側キーは Win+Shift+S (スクリーンショット)
- エンコーダ: 通常 PageUp / PageDown、ARROW 中 Ctrl+PageUp / PageDown (Combo は 6.7 節)

注意

Layer 4 と 5 はトラックボールから番号で参照 されているので、この 2 つの位置は固定しておく と管理しやすくなります。

8.1 Auto Mouse Layer と Scroll Layer (`roBa.keymap`)

```
&trackball {  
  automouse-layer = <4>; // 動かすと Layer 4 を一時的に有効化  
  scroll-layers = <5>; // Layer 5 が有効な間は移動をスクロールにする  
};
```

トラックボールを動かす



Layer 4 (MOUSE) が有効になる

J / K / L がクリックになる



最後の入力から `AUTOMOUSE_TIMEOUT_MS` 経過



Layer 4 が解除される

I を長押し (Layer 5) しながら
トラックボールを動かす



スクロールとして扱われる

TIP

MOUSE レイヤーは、文字キーを大量に上書きするより **クリックだけを必要な位置に置き、他は `&trans` にすると扱いやすくなります。** Auto Mouse 中の誤入力が気になるキーだけ `&none` にする設計も有効です。

`scroll-layers` は複数指定できます (例: `scroll-layers = <5 7>;`)。

8.2 roBa_R.conf の主な項目

トラックボール側（右）の `boards/shields/roBa/roBa_R.conf` で調整します。

設定	現在値	意味
<code>CONFIG_PMW3610_CPI</code>	400	カーソル感度。大きいほど速い
<code>CONFIG_PMW3610_CPI_DIVIDOR</code>	1	CPI をこの値で割る（微調整用）
<code>CONFIG_PMW3610_AUTOMOUSE_TIMEOUT_MS</code>	700	最後のトラックボール入力から Auto Mouse Layer を解除するまでの時間
<code>CONFIG_PMW3610_SCROLL_TICK</code>	16	スクロール 1 段あたりに必要な移動量。大きいほどスクロールが遅い
<code>CONFIG_PMW3610_ORIENTATION_180</code>	y	センサーの取り付け向き
<code>CONFIG_PMW3610_INVERT_X</code>	n	X 方向を反転
<code>CONFIG_PMW3610_INVERT_SCROLL_X</code>	y	横スクロール方向を反転
<code>CONFIG_PMW3610_MOVEMENT_THRESHOLD</code>	0	この移動量未満は無視（Auto Mouse の誤発動防止に使える）
<code>CONFIG_PMW3610_SMART_ALGORITHM</code>	y	センサーの追従性を改善するアルゴリズム
<code>CONFIG_PMW3610_POLLING_RATE_125_SW</code>	y	ポーリングレート 125Hz
<code>CONFIG_ZMK_STUDIO</code>	y	ZMK Studio を有効化

8.2 .conf の変更例 / 8.3 実験的な機能

例: Auto Mouse の継続時間を 1.5 秒にする

```
CONFIG_PMW3610_AUTOMOUSE_TIMEOUT_MS=1500
```

例: 精密操作の Snipe モードを使う

.conf のコメントアウトを外し、.keymap で Snipe レイヤーを指定します。

```
CONFIG_PMW3610_SNIPE_CPI=800  
CONFIG_PMW3610_SNIPE_CPI_DIVIDOR=4
```

```
&trackball {  
    automouse-layer = <4>;  
    scroll-layers = <5>;  
    snipe-layers = <7>;    // Layer 7 が有効な間は低感度になる  
};
```

注意

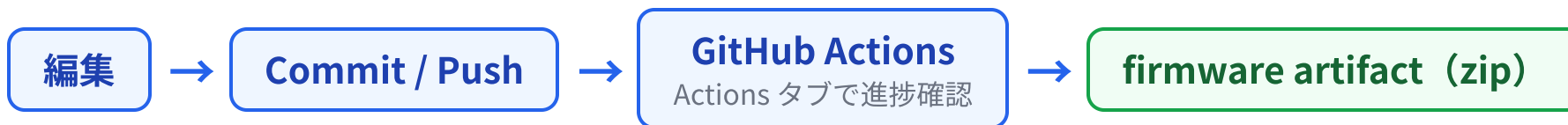
.conf を変更した場合は **ファームウェアの再ビルドと書き込み** が必要です。ZMK Studio からは変更できません。

8.3 トラックボールで矢印キー

roBa.keymap にはコメントアウトされた arrows { ... } ブロックがあります。有効にすると、指定レイヤー中のトラックボール移動を矢印キーに変換できます。必要なときにコメントを外して試してください。

9.1 GitHub Actions でビルドする

Keymap Editor で Save するか、GitHub / ローカル Git から commit & push すると `build.yml` が自動で走ります。



1. 自分のリポジトリの **Actions** タブを開く
2. 最新の実行が緑（成功）になっているか確認する
3. 実行を開くと **Artifacts** に `firmware` があるのでダウンロード
4. zip の中に右の UF2 が入っている

```
roBa_R-seeeduino_xiao_ble-zmk.uf2  
roBa_L-seeeduino_xiao_ble-zmk.uf2  
settings_reset-seeeduino_xiao_ble-zmk.uf2
```

NOTE

ビルドが赤（失敗）のときは、ログ末尾に `.keymap` の構文エラーの行番号が出ます。Editor 編集直後の失敗は、手書き部分（Combo の位置番号やカスタム Behavior）を疑ってください。

9.2 書き込む / 9.3 どちら側を書き換えるか

9.2 書き込み手順

1. 書き込みたい側の roBa を USB で PC に接続
2. リセットボタンを **素早く 2 回** 押す（またはキーマップの `&bootloader` を押す）
3. `XIA0-SENSE` という名前の USB ドライブとして認識される
4. 対応する UF2 ファイルをドライブにコピー
5. 自動的に再起動して書き込み完了

ロータリーエンコーダ側 (L, Peripheral)

↕ BLE

トラックボール側 (R, Central)

↕ USB / BLE

PC

9.3 どちら側を書き換えるか

変更内容	R (トラックボール側)	L (エンコーダ側)
<code>roBa.keymap</code> のみ	○	原則不要
<code>roBa_R.conf</code> (CPI / timeout)	○	○ 推奨
<code>roBa_L.conf</code> / overlay / shield	○	○
<code>west.yml</code> (ZMK バージョン更新)	○	○

TIP

キーマップは Central (R) が処理するので、通常のキー変更は **R 側だけ書き込めば反映** されます。

9.4 settings_reset は通常使わない

settings_reset は Bluetooth のペアリング情報や内部設定を **すべて消す** ためのファームウェアです。通常のキーマップ変更では不要です。

左右がつながらなくなった、PC とペアリングできなくなった、といったトラブル時に使います。

左右両方に settings_reset を書き込む



左右両方に通常の roBa_L / roBa_R を書き込む



PC の Bluetooth 設定から roBa を削除して再ペアリング

PART 4

日々の運用と困ったとき

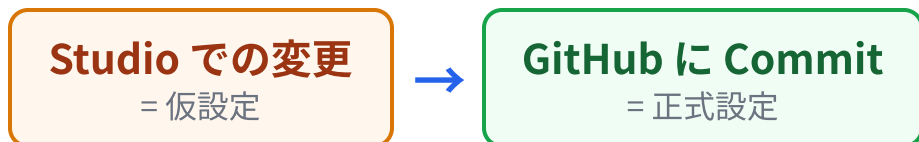
10. ZMK Studio ／ 11. 運用 Tips ／ 12. トラブルシューティング ／ 13. チートシート ／ 14. 参考資料

10. ZMK Studio との付き合い方

ZMK Studio (<https://zmk.studio/>) は、ブラウザから USB 経由で roBa に接続し、再ビルドなしでキーマップを変更できるツールです。この repo では R 側で有効 (`build.yaml` の `studio-rpc-usb-uart` と `CONFIG_ZMK_STUDIO=y`)。

10.1 使いどころ

「左クリックを J と F のどちらに置くか試したい」のような **短時間の試行** に向いています。



良い変更が見つかったら **必ず Keymap Editor で roBa.keymap にも反映** してください。二重管理が面倒なら、編集を Keymap Editor に一本化するのが最も安全です。

10.2 Studio でできないこと

- `.keymap` への Export / Import (2026-09 時点で Planned)
- Combo / Macro / カスタム Behavior の編集
- `.conf` の変更 (CPI、Auto Mouse timeout)
- 変更履歴の管理

10.3 Restore Stock Settings / 10.4 レイヤー名

10.3 Restore Stock Settings の意味

「工場出荷時に戻す」というより、

NOTE

現在のファームウェアに焼き込まれている `.keymap` の内容を、Studio の Runtime Keymap に再ロードする 操作です。

GitHub 側の `.keymap` を更新して新しい UF2 を書き込んだのに反映されないときは、これを実行します。

10.4 Studio でレイヤー名を見やすくする

Studio はレイヤーのノード名（`FUNCTION` など）を表示します。`display-name` を付けると任意の名前にできます。

```
layer_6 {  
    display-name = "SYSTEM";  
    bindings = < ... >;  
};
```

11.1 標準フロー — 小さく変えて、数日使って、判断する



11.2 変更は小さく、メッセージは具体的に

おすすめしない: update keymap
おすすめ: mouse: move left click to J
arrow: add Home/End
base: swap Backspace and Delete
trackball: extend automouse timeout to 1500ms

後で「どの変更が使いにくさの原因だったか」を追いやしくなります。

レイヤー全体を一気に作り替えると、良し悪しの判断が付きません。

11.3 Tag / 11.4 ブランチ / 11.5 キーマップ図

11.3 安定版に Tag を付ける

```
git tag -a v1.0 -m "first stable roBa keymap"  
git push origin v1.0
```

GitHub の **Releases** でその Tag から Release を作り、その時点の UF2 を添付しておく、数年後でも再ビルドせずに書き戻せます。

11.4 大きな変更はブランチで

Home Row Mods 導入、レイヤー全面再設計、ZMK バージョン更新など。

```
git switch -c experiment/home-row-mods
```

Keymap Editor の Branch 選択でこのブランチを選べば GUI から commit できます。うまくいったら `main` に merge、駄目なら捨てます。

11.5 キーマップ図を更新する

1. **Actions** タブ → **Draw Keymap**
2. **Run workflow** を押す
3. 図を更新する commit が自動で追加される

push のたびに自動実行したい場合は `draw.yml` 内のコメントアウトを外します。

11.6 公式更新を取り込む / 11.7 KEYMAP_NOTES.md

11.6 公式更新を取り込む

GitHub の **Sync fork** ボタン、またはローカル Git で取り込みます。

```
# 復帰点を作ってから
git tag -a before-upstream-sync-$(date +%Y%m%d) -m "backup before upstream sync"
git push origin --tags

git fetch upstream
git log --oneline main..upstream/main # 何が変わるか確認
git switch main
git merge upstream/main
```

11.7 KEYMAP_NOTES.md を残す

キーマップだけでは「なぜこの位置にしたか」が残りません。理由を書いておくと、半年後の自分に役立ちます。

Mouse layer

- J: Left Click / K: Middle Click / L: Right Click
- J は右手人差し指で押しやすいため左クリックにした

ARROW layer

- HJKL は矢印にせず、Vim と競合しない配置を試している

NOTE

11.6 で自分と公式の両方の `roBa.keymap` が変わっていると Conflict します。慎重に運用するなら `boards/build.yaml` `config/west.yaml` の更新だけ取り込み、`roBa.keymap` は自分のものを維持する方針もあります。

12. トラブルシューティング (1/2)

`.keymap` を変更したのに反映されない

最有力候補は **ZMK Studio の Runtime Keymap が残っている** ことです。

1. Actions が成功していて、新しい UF2 を本当に書き込んだか
2. **R 側**（トラックボール側）に書き込んだか
3. ZMK Studio で **Restore Stock Settings** を実行

注意

Restore の前に、Studio 側にしかない未移行の設定がないか確認してください。

トラックボールを動かしても MOUSE レイヤーにならない

1. `roBa.keymap` に `&trackball { automouse-layer = <4>; scroll-layers = <5>; }` があるか
2. Layer 4 が本当に MOUSE レイヤーか（レイヤー挿入で番号がずれていないか）
3. Layer 4 のクリック位置が `&mkp MB1` などになっているか
4. Studio の古い Runtime Keymap が残っていないか（Restore Stock Settings）
5. R 側に最新の UF2 を書き込んだか
6. 切り分けのため `CONFIG_PMW3610_AUTOMOUSE_TIMEOUT_MS=1500` にして試す

12. トラブルシューティング (2/2)

Jを押すと左クリックではなく j が入力される

- A. どこかのキーで &mo 4 を押しながら J → 左クリックになる
→ MOUSE レイヤー自体は正しい。
Auto Mouse 側 (&trackball の設定 / timeout) を疑う
- B. &mo 4 を押しながら J でも j が入力される
→ Layer 4 のキーマップ設定、またはレイヤー番号のずれを疑う

Mod-Tap / Layer-Tap の誤爆が多い

- 長押しのつもりがタップ → `tapping-term-ms` を短く、`flavor = "hold-preferred"`
- タップのつもりが長押し → `tapping-term-ms` を長く、`require-prior-idle-ms`
- 連打がリピートしない → `quick-tap-ms = <150>;`

ビルドが失敗する

- Actions のログ末尾のエラー行番号を `roBa.keymap` で確認
- 各レイヤーの `bindings` のキー数が **43 個** になっているか
- Combo の `key-positions` が **0~42** の範囲か
- カスタム Behavior 名や Macro 名のタイプミスがないか

左右が繋がらない / PC とペアリングできない

9.4 節の手順で `settings_reset` を左右両方に書き込み、通常ファームウェアを書き直してから再ペアリングします。

13. チートシート

やりたいこと	手順
キーを変える	Keymap Editor → Save → Actions → R 側 UF2
Auto Mouse timeout / CPI を変える	<code>roBa_R.conf</code> → commit → Actions → R / L 両方の UF2
Combo / Macro を足す	<code>roBa.keymap</code> を手書き → commit → Actions → R 側 UF2
<code>.keymap</code> 更新が反映されない	ZMK Studio → Restore Stock Settings
書き込みモードに入る	リセット 2 回押し、または <code>&bootloader</code> キー
安定版を保存	<code>git tag -a v1.0 -m "stable keymap" && git push origin v1.0</code>
公式更新を確認	<code>git fetch upstream && git log --oneline main..upstream/main</code>
キーマップ図を更新	Actions → Draw Keymap → Run workflow
別の roBa に移植	安定版 UF2 を R / L に書き込み → 再ペアリング

14. 参考資料

roBa

- roBa 公式リポジトリ
<https://github.com/kumamuk-git/roBa>
- roBa v3 ビルドガイド
https://github.com/kumamuk-git/roBa/blob/main/doc/v3/buildguide_v3.md
- roBa 公式 ZMK config
<https://github.com/kumamuk-git/zmk-config-roBa>
- PMW3610 ドライバ (roBa 用フォーク)
<https://github.com/kumamuk-git/zmk-pmw3610-driver>

ツール

- Keymap Editor: <https://nickcoutsos.github.io/keymap-editor/>
- ZMK Studio: <https://zmk.studio/> • <https://zmk.dev/docs/features/studio>
- keymap-drawer: <https://github.com/caksoylar/keymap-drawer>

ZMK ドキュメント

- Keymaps & Behaviors: <https://zmk.dev/docs/keymaps>
- キーコード一覧: <https://zmk.dev/docs/keymaps/list-of-keycodes>
- Hold-Tap (Mod-Tap / Layer-Tap の詳細) :
<https://zmk.dev/docs/keymaps/behaviors/hold-tap>
- Combos: <https://zmk.dev/docs/keymaps/combos>
- Macros: <https://zmk.dev/docs/keymaps/behaviors/macros>
- Mouse / Pointing:
<https://zmk.dev/docs/keymaps/behaviors/mouse-emulation>
- Configuration Overview: <https://zmk.dev/docs/config>

まとめ

ZMK Studio は試す場所、GitHub の `roBa.keymap` は残す場所。

- 日常の変更は **Keymap Editor** → **Save (= commit)** → **Actions** → R 側に UF2
- Combo / Macro / トラックボール設定は **手書き** で `.keymap` / `.conf` へ
- 反映されないときは **ZMK Studio** → **Restore Stock Settings**
- 変更は **小さく・具体的なメッセージ**で、安定したら **Tag**